then replacing .count() with .size() and calling trips.size repeatedly, notice how no SQL queries are performed to fetch the value. driver.trips.size is equivalent to reading the field drivers.trips_count directly, which was populated earlier.

You've now seen a variety of ways to use Caches to improve your query performance.

Another type of poor performance can be conducting calculations in Ruby code. PostgreSQL has a rich variety of calculation methods available and may be a better place to conduct this work. These methods are available to use with Active Record.

Performing Aggregations In the Database

One opportunity to improve Active Record performance is to perform calculations in PostgreSQL instead of in Ruby and Active Record code, by using built-in SQL aggregate functions.

Database aggregate operations will generally provide better calculations faster due to fewer round trips between the application and the database.

PostgreSQL aggregate functions and Active Record equivalents are displayed below.

	Active Record	SQL
Count	.count()	COUNT()
Average	.average()	AVG()
Minimum	.minimum()	MIN()
Maximum	.maximum()	MAX()
Sum	.sum()	SUM()

Table 1—Aggregations Comparison

Equivalent Aggregations in Active Record and SQL

Active Record support is provided in the ActiveRecord::Calculations[30] module. For PostgreSQL, refer to documentation on *Aggregate Functions*.[31]

PostgreSQL has more Aggregate Functions beyond those supported by Active Record. PostgreSQL even allows users to make their own aggregate functions called *User-Defined Aggregates*.[32]

30. https://api.rubyonrails.org/classes/ActiveRecord/Calculations.html
31. https://www.postgresql.org/docs/current/functions-aggregate.html
32. https://www.postgresql.org/docs/current/xaggr.html